



**TSMC CL013G Process
High-Speed Synchronous Dual-Port SRAM
(SRAM-DP-HS) Generator
User Manual**

September 2002

Release 2.1

Copyright © 1997-2002 Artisan Components, Inc. All rights reserved.
Printed in the United States of America.

Part Number UG-SD-TSM014-2.1/13

Artisan Components is a registered trademark and Artisan, Process-Perfect, Universal Test Interface, SAGE, SAGE-HS, SAGE-X, SAGE-Modeler, Flex-Repair, ArtNuvo and ElectroArt are trademarks of Artisan Components, Inc. Artisan acknowledges the trademarks of other organizations for their respective products or services mentioned in this document.

Artisan reserves the right to make changes to any products and services described herein, at any time without notice in order to make improvements in design, performance, or presentation and to provide the best possible products and services. Customers should obtain the latest specifications before referencing any information, product, or service described herein, except as expressly agreed in writing by an officer of Artisan.

Artisan does not assume any responsibility or liability arising out of the application or use of any products or services described herein, except as expressly agreed to in writing by an officer of Artisan; nor does the purchase, lease, or use of a product or service from Artisan convey a license under any patent rights, copy rights, trademark rights, or any other intellectual property rights of Artisan or of third parties.

Artisan Components, Inc., 141 Caspian Court, Sunnyvale, CA 94089, USA

Unpublished – rights reserved under the copyright laws of the United States.

Table of Contents

Preface

Revision History	vii
Customer Support	vii
Typographic Conventions	viii

Chapter 1

Introduction and Installation	1-1
Introduction	1-3
Performance, Power and Reliability	1-3
Designed to Meet Tight Market Windows	1-3
Process-Specific Tuning	1-3
Installing the TSMC 0.13-micron Dual-Port SRAM Generator	1-4
System Requirements	1-4
Installing the SRAM Generator	1-4
Directory Structure	1-6

Chapter 2

Using the Dual-Port SRAM Generator	2-1
Dual-Port SRAM Generator Features	2-3
Using the Artisan Generator Graphic User Interface (GUI)	2-4
Starting the GUI	2-4
Specification Files	2-6
Setting Generic Parameters	2-7
Setting Advanced Options	2-8
Setting View-Specific Parameters	2-8
Synopsys Model Parameters	2-9
TLF Model Parameters	2-9

Generating Views	2-10
Generating Single Views	2-10
Generating Multiple Views	2-10
Generating PostScript Datasheets	2-12
Generating Datasheets from the Generator GUI	2-12
Generating Datasheets from the Command Line	2-12
Generating Configuration Parameter Files	2-13
Generator Cancel Option	2-14
Log File	2-14
ASCII Datable Pane	2-15
Balloon Help	2-16
Online Documentation	2-16
Relative Footprint Pane	2-16
Exiting the GUI	2-16
Using View Commands to Generate Views	2-17
Using View-Specific Options to Generate Multiple View Commands	2-17
Generator View Options	2-19
Syntax	2-19
Global Options	2-19
Generic Options	2-20
Advanced Options	2-22
View-Specific Options	2-24

Chapter 3

Dual-Port Synchronous SRAM Generator Architecture	3-1
Synchronous Generator Architecture	3-3
SRAM Description	3-3
Block Parameters	3-5
Core Address Mapping	3-7
Power Structure	3-10
SRAM Current	3-10
Power Distribution Methodology	3-11

Supply Connections to Power Rings	3-12
Top Metal Layer	3-14
I/O Connections	3-14
Timing Specifications	3-15
Clock Timing	3-15
Mission-Mode Timing	3-18
Timing Parameters	3-20
Characterization Environments	3-21
Timing Derating	3-21

Chapter 4

Using the Dual-Port SRAM Generator Views	4-1
Using the Generated Views	4-3
Tool Verification	4-3
Using the Verilog Model	4-4
Using the VHDL Model	4-5
Using the Synopsys Model to Generate SDF	4-7
Using the Pearl Model to Generate SDF	4-8
Using the PrimeTime Model to Generate SDF	4-9
Loading the VCLEF Description into Silicon Ensemble	4-10
Using Apollo with Artisan SRAM	4-11
Loading the VCLEF Description into the Milkyway Database	4-11
Loading the GDSII Layout into the Milkyway Database	4-12
Loading the GDSII Layout into a DFII Library	4-13
Using the LVS Netlist	4-14
Using Hierarchical LVS	4-15
Index	Index-1

Preface

Revision History

This section provides the release history for the TSMC CL013G Process SRAM-DP-HS Generator User Manual.

Part Number	Release Number	Date of Release	Updates
UG-SD-TSM014-1.0/13	1.0	March 2001	• Initial Release
UG-SD-TSM014-1.1/13	1.1	April 2001	• GUI shown on p.2-5
UG-SD-TSM014-1.2/13	1.2	April 2001	• Corrected total memory bits range on p.3-6
UG-SD-TSM014-1.3/13	1.3	September 2001	• New operating system text added
UG-SD-TSM014-2.0/13	2.0	December 2001	• Tool versions updated
UG-SD-TSM014-2.1/13	2.1	September 2002	• Recharacterization

Customer Support

For all customer service or technical support questions, please visit the Artisan Components Web site at www.artisan.com and click on Customer Support.

You may also contact Artisan by telephone or email, using the following information:

■ United States and North America	877-ARTILIB (877-278-4542)
■ International	408-548-3298
■ Email	support@artisan.com

Typographic Conventions

Typographic conventions are used to assist you in distinguishing special notations, values, and elements described in this manual.

Visual Cue	Meaning
(Bullet) ■	Bulleted list of important items.
(Pointing Hand) 	Important information or explanation.
Courier Type	Commands typed on the keyboard, either examples or instructions. This shows variable and command names within regular text.
< <i>italic type</i> > <i>italic type</i>	Variable names you select, such as file and directory names are enclosed within angle brackets (< >). Italic type is used to show variable values, file, and directory names.
(Ellipsis) ...	Indicates that commands or options may be added.
<i>Italic Type with Initial Capital Letters</i>	Document, chapter, section, and reference manual names.



1

Introduction and Installation

Introduction

Artisan Components is the world's leading supplier of intellectual property (IP) components, providing highly optimized, high quality IP solutions to the world's leading semiconductor foundries, integrated device manufacturers, ASIC vendors, and IC design teams. With hundreds of licensees, over two thousand production-proven designs, and the largest partner network of IP, EDA, and service providers, Artisan's products have become the industry standard for silicon quality and ease-of-use.

Performance, Power and Reliability

Available for 0.10 through 0.25-micron semiconductor processes, Artisan's Process-Perfect™ memory generators, and standard cell and I/O cell libraries have been optimized to achieve the full-speed, high density, and low power potential of each semiconductor process. Each product is verified using the industry's most comprehensive QA and validation procedures. Products are delivered ready-to-use, with a complete set of views and models supporting leading design tools.

Artisan's proprietary library development methodology has been proven in over 100 different manufacturing processes, and in tens of millions of production parts.

Designed to Meet Tight Market Windows

Artisan's Process Perfect methodology enables IC design teams to reduce the time to market for new system-chips by ensuring that each IP component is designed to achieve the best combination of performance, power, density, and yield for each customer's semiconductor manufacturing process.

In addition, all Artisan products are delivered with a complete set of views and models to support industry leading design tools. Artisan provides designers with ready-to-use solutions suitable for even the most aggressive designs.

Process-Specific Tuning

Process-specific tuning begins after analyzing a customer's design objectives and the capabilities of the silicon process technology. Using intensive design procedures, the process-specific tuning includes architectural, electrical, and geometric optimizations to achieve the best possible area, performance, and power.

Installing the TSMC 0.13-micron Dual-Port SRAM Generator

This section provides information about the operating system and installation requirements for your memory generator.

System Requirements

Artisan memory generators are supported by the following operating systems:

- Solaris 2.6 (also called SunOS 5.6) and the latest SunOS patches, or Solaris 7 (also called SunOS 5.7) and the latest SunOS patches
- HP-UX version 11

 If your operating system does not meet the requirements stated above, you may receive an error message when running the memory generator. In this case, the generator will not function until you upgrade your operating system.

To determine the name and version of your SunOS or HP-UX operating system, enter the following command:

```
uname -a
```

Installing the SRAM Generator

You must determine where you want to install the generator on your system. In this manual, *<install_dir>* refers to the directory you choose for installation.

 When copying the installation files, you should create a new directory. Overwriting an existing generator directory may corrupt the generator installation.

Make sure you have enough disk space available for your installation. The generator installation requires approximately 40 megabytes.

To install the generator from your CD-ROM onto your hard disk:

Change to the installation directory:

```
cd <install_dir>
```

Copy the installation files from the CD-ROM to the installation directory on your hard disk:

```
cp /cdrom/<cdrom_name>/* .
```

where <cdrom_name> is the name of your CD-ROM.

Uncompress the installation files:

```
gunzip <install_files>.tgz
```

where <install_files> represents the generator installation files in your installation directory.

Extract the installation files:

```
tar -xvf <install_files>.tar
```

where <install_files> represents the uncompressed installation files.

Directory Structure

Installing the TSMC 0.13-micron dual-port SRAM generator produces the following directory structure in *<install_dir>*.

aci/Executable/

- bin/* This directory contains the generator executable and platform-specific directories.
- lib/* This directory contains technology files, library files, executables, and subdirectories.
 - vhdl_lib/* This directory contains the VHDL library file, *artisan_lib.vhd*.
- doc/* This directory contains generator documentation.

 The Artisan Generator GUI is written in Java. For your convenience, the generator source tree includes copies of all required JRE distributions.



2

Using the Dual-Port SRAM Generator

Dual-Port SRAM Generator Features

Artisan's memory generators provide integrated circuit designs with the highest levels of density, speed, and power. The TSMC 0.13-micron dual-port SRAM generator offers a comprehensive set of output views, with specific parameters, for a synchronous SRAM.

You can run the generator in two ways:

- Invoke the graphical user interface (GUI)
- Run the generator from the command line

You can generate the following views:

- PostScript Datasheet
- ASCII Datatable
- Verilog Model
- VHDL Model
- Synopsys Model
- PrimeTime Model
- TLF Model
- VCLEF Footprint
- GDSII Layout
- LVS Netlist

This chapter describes both methods of generating views, and their associated options and parameters.

Using the Artisan Generator Graphic User Interface (GUI)

The Artisan generator GUI is an interface that allows you to configure all generator parameters and generate all views. The output views along with a log file are placed in the current working directory.

The following sections describe how to start the GUI, configure generic and view-specific parameters, and generate views.

Starting the GUI

This manual assumes you have added `<install_dir>/aci/ra2sh/bin` to your UNIX search path. If you do not wish to do this, preface all generator commands with `<install_dir>/aci/ra2sh/bin.`

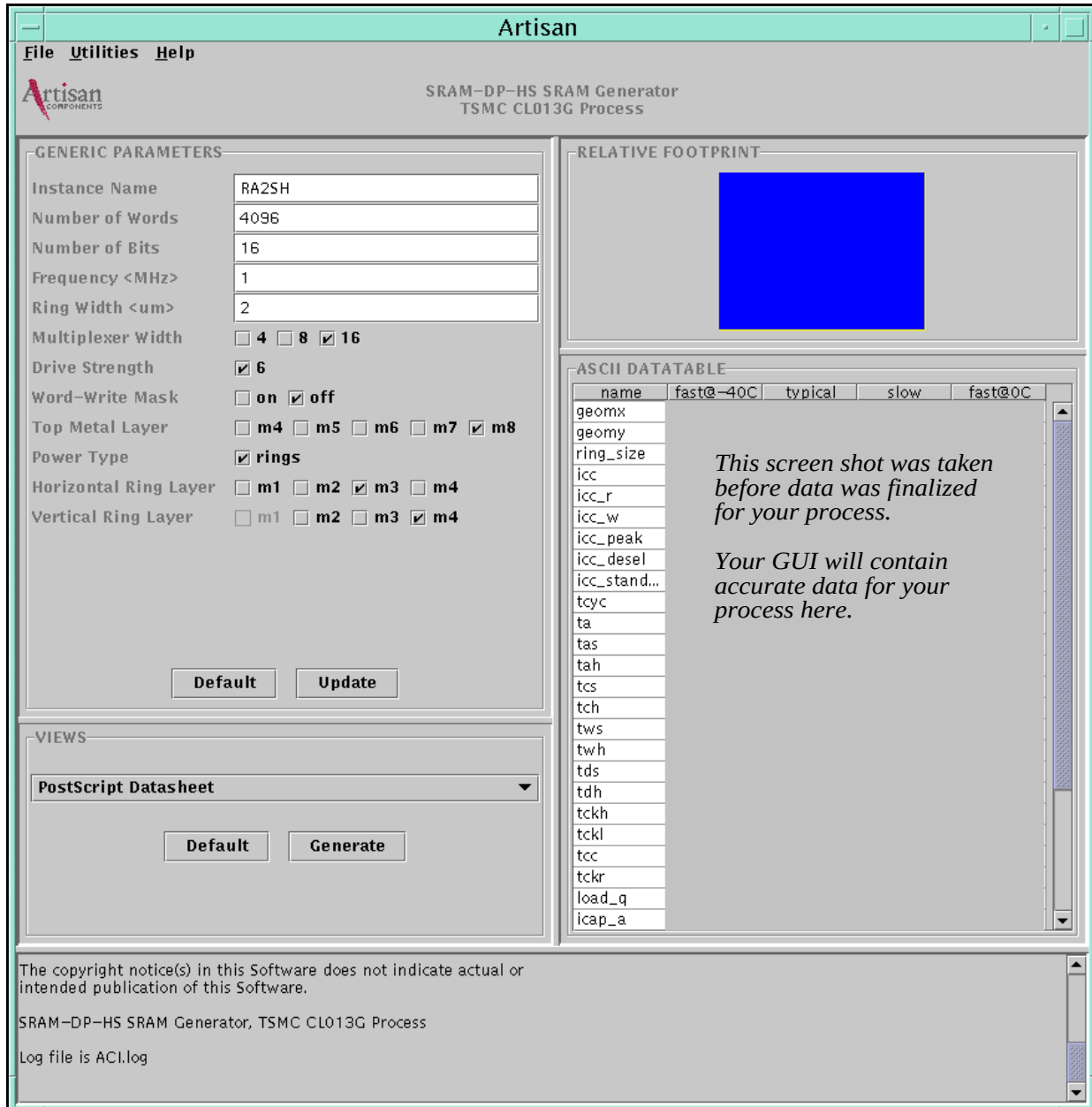
To start the GUI from the shell, type:

```
% cd <working_dir>
% ra2sh
```

where `<working_dir>` refers to the directory where you choose to run the generator. Generator output files are created in the current directory; therefore, Artisan strongly recommends that you run the generator in a working directory that is different from the source directory `<install_dir>`.

The GUI appears on your screen as shown in Figure 2-1. The panes of the GUI may be resized by clicking on the pane border and dragging it to the desired position.

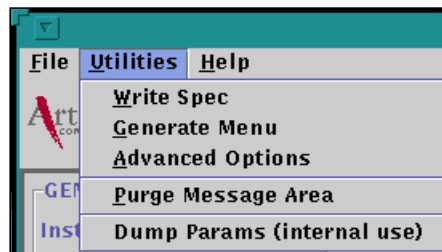
Figure 2-1. TSMC 0.13-micron Dual-Port SRAM Generator GUI



Specification Files

When you select the Utilities pull-down menu, then select *Write Spec*, a specification file is generated and placed in the current working directory *<working_dir>*. The generated specification file is an ASCII text file that contains all generic, advanced, and view-specific parameter values set at the time the file is generated. The name of the generated specification file is *<instance_name>.spec*, where *instance_name* is the name shown in the *Generic Parameters* pane of the GUI at the time the specification file is generated. This file may be edited and used for subsequent runs. Figure 2-2 shows a sample Utilities pull-down menu.

Figure 2-2. Sample Utilities Pull-Down Menu



You may also create your own specification file using any ASCII text editor. The format for this file is a list of the options you want to apply to your model. When using options in a specification file, do not place a dash "-" in front of the option name. You may use either a space or an equal sign "=" between the option name and the value. Figure 2-3 is an example of a simple specification file that includes a few options you may want to use frequently.

Figure 2-3. Sample Specification File

```
prefix MY_  
instname RA2SH  
mux 8  
words 256  
vclef-fp.site_def=off  
vclef-fp.inst2ring=blockages  
top_layer=met8  
write_mask=off
```

Name your specification file, and save it. Your specification file may then be used to configure the generated instance. Launch the GUI with the parameter values from your specification file as the defaults:

```
% ra2sh -spec <spec_file>
```

where <spec_file> is the name of your specification file.

You may supply any combination of options and specification files on the command line. On the command line, use the dash "-" in front of the option. In the case of duplicate options, the last option set takes precedence.

The specification file may also be used with the generator shell commands. Refer to the "Generator View Options" section on page 2-19.

Setting Generic Parameters

The *Generic Parameters* pane of the GUI contains input fields for generic parameters. Refer to Figure 2-1 "TSMC 0.13-micron Dual-Port SRAM Generator GUI" on page 2-5. Generic parameters apply to all the view generators.

You can change the value of a generic parameter by typing the new value in the input field or by selecting the button corresponding to the desired option value. See the "Generator View Options" section on page 2-19 for the default values and ranges of the generic parameters.

When you want to submit the values of the generic parameters and update the ASCII datatable, click on the *Update* button in the *Generic Parameters* pane.

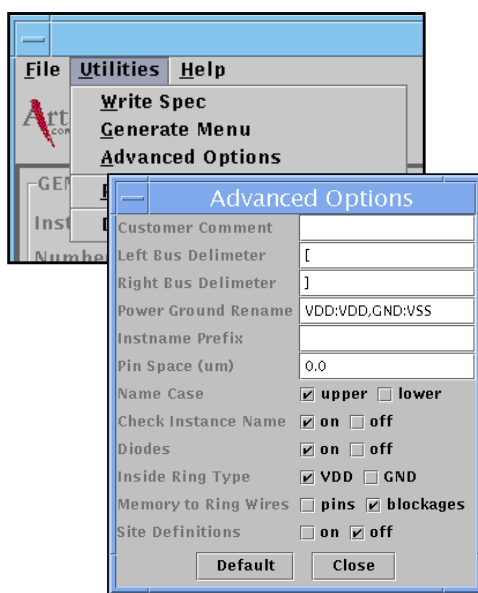
To reset the generic parameters to their default values, click on the *Default* button in the *Generic Parameters* pane.

For example, to generate views for a synchronous dual-port SRAM, set 256 words, 16 bits, multiplexer width 8, and leave all other parameters set to their default values. Enter "256" in the *Number of Words* field, "16" in the *Number of Bits* field, and select the button corresponding to "8" for multiplexer width. You can select the view you want from the "Views" pane, then click on *Generate*, or click on *Utilities* and select *Generate menu*.

Setting Advanced Options

To view the *Advanced Options* pop-up window, select the *Utilities* pull-down menu. Refer to Figure 2-2 "Sample Utilities Pull-Down Menu" on page 2-6. Then select *Advanced Options*. The *Advanced Options* pop-up window displays the advanced options that you can specify. Sample advanced options are shown in Figure 2-4. If advanced options are not included in the generator, a message is displayed in the *Advanced Options* window. See the "Advanced Options" section on page 2-22 for the default values and ranges of the Advanced Options.

Figure 2-4. Sample Advanced Options Pop-up Window



Setting View-Specific Parameters

The *Views* pane of the GUI provides a pull-down menu that displays the views you can generate. Refer to Figure 2-1 "TSMC 0.13-micron Dual-Port SRAM Generator GUI" on page 2-5.

To select a view, click on the corresponding option in the pull-down menu. If the view is not available, a message is displayed in the *Views* pane. When you click on a view and click *Generate*, the view-specific parameters appear in the message pane at the bottom of the GUI.

For each view, you can click *Default* to set the view parameters to their default values. As you move from view to view, the parameter values for each view are retained.

Refer to the "Generator View Options" section on page 2-19 for more information about the default values and ranges of the view-specific parameters. Additional details about view-specific parameters are provided in the following sections.

Synopsys Model Parameters

Library Name This option specifies the Synopsys library name. The default name is USERLIB; you must specify this option.

The typical and slow Synopsys models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

TLF Model Parameters

Library Name This option specifies the TLF library name. The default name is USERLIB; you must specify this option.

The typical and slow TLF models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

Generating Views

You can generate a single view through the *Views* pane in the generator GUI, or select all or a partial list of views to generate through the *Generate Menu*.

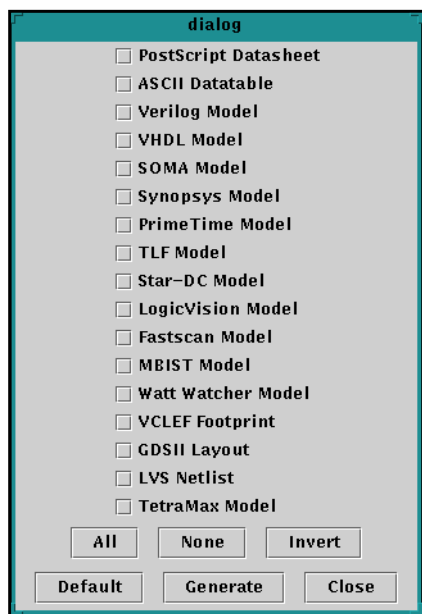
Generating Single Views

To generate a single view, select the view you want from the *Views* pull-down menu then click on the *Generate* button in the *Views* pane. The corresponding view is generated and placed in the current working directory *<working_dir>*. A list of available views and their output files is shown in Table 2-2 "View Commands".

Generating Multiple Views

To generate all views or a selection of views, select the *Utilities* pull-down menu, as shown in Figure 2-2 "Sample Utilities Pull-Down Menu" on page 2-6. Then select *Generate Menu*, such as the sample shown in Figure 2-5 "Sample Generate Menu." The *Generate Menu* pop-up window shows a list of views that you can generate. The views provided for your generator may be different from what is shown below.

Figure 2-5. Sample Generate Menu



When this menu is first opened, all views are selected. Click on the box corresponding to a view to toggle between selecting and de-selecting the view. When a view is selected, the box contains a check mark. When you click on the *All* button, all views listed are selected. When you click on the *None* button, all views listed are de-selected. When you click on the *Invert* button, the selection of views is inverted.

When you click on the *Default* button, the view-specific parameters for the selected views are initialized to their default values. When you click on the *Generate* button, all selected views are generated and placed in the current working directory *<working_dir>*.

If you close the *Generate-Menu* pop-up window and reopen it during the same GUI session, the most recently selected list is recalled.

Generating PostScript Datasheets

You can generate an A-size PostScript datasheet for selected parameters. Instead of using the default values for mux, bits, words, horizontal or vertical metals, or top layer metals you can change these parameters and save the result in a datasheet file. You can create datasheets from the generator GUI or from the command line.

Generating Datasheets from the Generator GUI

Generate a PostScript datasheet from the generator GUI by selecting "Postscript Datasheet" in the Views pane in the generator GUI. Click on Generate. The output datasheet, *RA2SH.ps*, is placed in the current directory, *<working_dir>*.

Generating Datasheets from the Command Line

Use the following command to generate a PostScript datasheet from the command line.

```
postscript
```

The following example demonstrates using the command line to create a datasheet with selected and default parameters.

EXAMPLE

```
ra2sh postscript -words 256 -bits 16 -mux 8
```

The command in this example generates a PostScript datasheet for a synchronous dual-port SRAM with 256 words, 16 bits, mux 8, with all other parameters set to their default values. The output datasheet, *RA2SH.ps*, is placed in the current directory, *<working_dir>*.

Generating Configuration Parameter Files

When you generate an instance from the GUI, a report containing configuration information appears in the message pane at the bottom of the GUI. Configuration information for that instance also outputs to a file in the current directory, the filename of which is indicated in that instance's message pane. The resulting file contains the same configuration details that appeared in the GUI message pane, with the addition of identifiers (name, version, rom code file, customer comment string) at the top of the file. The file stores the configuration information so that configuration details for the instance can be easily accessed by using any text reader.

The file will be overwritten each time you run the GUI for a particular instance. In order to save each file, you must change the text string in the GUI *Instance Name* field.

EXAMPLE

```
* name:          SRAM-SP-HS SRAM Generator
*                Artisan 013um Process
* version:       2001Q4V0
* comment:       020322_6:38_03
* configuration: -instname RA1SHR02 -words 4096 -bits 16
                -frequency 1 -ring_width 2 -mux 16 -drive 6 -write_mask off
                -wp_size 8 -top_layer met8 -power_type rings -horiz met3 -
                vert met3 -cust_comment "020322_6:38_03" -left_bus_delim
                "[" -right_bus_delim "]" -pwr_gnd_rename "VDD:VDD,GND:VSS"
                -prefix "" -pin_space 0.0 -name_case upper -check_instname
                on -diodes on -inside_ring_type VDD
```

For more information on the customer comment string, see "Advanced Options" on page 2-22.

Generator Cancel Option

The GUI includes the capability of cancelling a generate action. When a view is being generated, a pop-up window displays a message stating which view is being generated, and a *Cancel* button. Click on the *Cancel* button to cancel generating the current view. If you are generating a list of views, the current view is cancelled, and the remaining views are not generated.

Log File

A log file containing a record of executed shell commands and output messages is placed in the same working directory. The name of the log file is shown in the message pane of the GUI along the bottom of the GUI frame. The usual name for this file is ACI.log. Refer to Figure 2-1 "TSMC 0.13-micron Dual-Port SRAM Generator GUI" on page 2-5.

When a view is generated or parameters are initialized to default values, a message is recorded in the log file and shown in the message pane.

You can clear the message pane by selecting the *Utilities* Menu, then selecting *Purge Message Area*. Although the message pane clears, the log file retains a full record of messages.

By default, each time the GUI is invoked, the log file created for that run overwrites the existing log file. You may choose to keep the existing log file(s) and create a new log file for the current session. To launch the GUI, keep existing logs, and create a new log file:

```
% ra2sh -keeplogs
```

The next log file is an incremental name generated by the system. In this example the second log file name is ACI.log.1.

ASCII Datable Pane

The *ASCII Datable* pane of the GUI displays parameters in the most recently generated ASCII datasheet. Figure 2-1 "TSMC 0.13-micron Dual-Port SRAM Generator GUI" on page 2-5 shows the GUI containing the ASCII datatable. Figure 2-6 shows a sample ASCII datatable pane. The characterization corners shown in this example may not be exactly the same as those in the TSMC 0.13 μ m process dual-port SRAM generator GUI.

Figure 2-6. Sample ASCII Datable Pane



The screenshot shows a window titled "ASCII DATABLE" containing a table with 5 columns: name, fast, typical, slow, and wide. The table lists various parameters and their values for different corners. A vertical scrollbar is on the right side of the table.

name	fast	typical	slow	wide
geomx	585.820	585.820	585.820	585.820
geomy	762.035	762.035	762.035	762.035
ring_size	5.200	5.200	5.200	5.200
icc	0.091	0.078	0.068	0.055
icc_r	0.083	0.071	0.061	0.049
icc_w	0.100	0.086	0.075	0.060
icc_peak	210.924	148.384	81.601	61.264
icc_desel	0.000	0.000	0.000	0.000
tcyc	0.816	1.181	1.952	2.340
ta	0.781	1.191	1.946	2.386
tas	0.237	0.362	0.614	0.704
tah	0.053	0.069	0.093	0.082
tcs	0.272	0.357	0.560	0.654
tch	0.000	0.000	0.000	0.000
twc	0.270	0.360	0.543	0.696
twh	0.000	0.000	0.000	0.000
tds	0.130	0.195	0.369	0.484
tdh	0.000	0.000	0.000	0.000
thz	0.430	0.582	0.908	1.020
tlz	0.382	0.518	0.798	0.969
tckh	0.083	0.108	0.183	0.120
tckl	0.112	0.167	0.281	0.361
tckr	4.000	4.000	1.828	1.709
load_q	0.271	0.358	0.531	0.607
icap_a	0.053	0.052	0.052	0.051
icap_d	0.004	0.004	0.004	0.003
ican_clk	0.264	0.257	0.229	0.230

When you invoke the GUI, the ASCII datasheet for the instance with default values is displayed in the *ASCII Datable* pane. When you change the generic parameter values in the GUI, and click on the *Update* button in the *Generic Parameters* pane, the ASCII datatable updates to reflect the new datasheet values.

You can resize the columns in the ASCII datatable by clicking on the column border and dragging it to the desired width. The columns can be rearranged by clicking on the header cell of a column and dragging it to the desired position.

The units for parameters in the ASCII datatable are listed in Table 2-1.

Table 2-1. ASCII Datatable Units

Parameters	Units
Geometry	Microns
Current-consumption	Milliamperes
Timing	Nanoseconds

Balloon Help

The GUI contains balloon help messages that assist in navigating and interpreting the generator. The balloon help messages appear as your mouse pauses over an active area such as ASCII datatable parameters, generic and view-specific parameters, and action buttons.

Online Documentation

The *Help* pull-down menu displays a list of documents that are shipped with the GUI and can be viewed online. When you select a document the Adobe PDF reader, *acroread*, launches and the document opens. If the document does not display properly, ask your system administrator to ensure that *acroread* is available to you and is in your path.

Relative Footprint Pane

The *Relative Footprint* pane of the GUI shows how the aspect ratio of the SRAM changes as the words, bits, and mux parameters are varied. Refer to Figure 2-1 "TSMC 0.13-micron Dual-Port SRAM Generator GUI" on page 2-5. The instance without a power ring is shown in the darker color. The power ring is shown in the lighter color. When you change a generic parameter and press return, the relative footprint is automatically updated.

Exiting the GUI

To exit the GUI, select the *File* pull-down menu, then select *Exit*.

Using View Commands to Generate Views

You can run the TSMC 0.13-micron dual-port SRAM generator directly from the command line using the generator View commands. A View command instructs the generator to produce the selected view.

You may use more than one View command on your command line. The specification file and individual option selections apply to all of the views selected for the run.

To call the generator directly from the shell, type:

```
% cd <working_dir>
% ra2sh <View_Command> [-spec <filename>][-options]...
```

where `<working_dir>` refers to the directory where you choose to run the generator. Generator output files are created in the current directory; therefore Artisan recommends that you run the generator in a working directory `<working_dir>` that is different from the source directory `<install_dir>`.

For example:

```
% ra2sh vclef-fp -inst2ring pins -mux 8 ...
% ra2sh vclef-fp gds2 synopsys tlf -mux 8 ...
```

Using View-Specific Options to Generate Multiple View Commands

Certain options are view-specific, which means that they only apply to certain View commands. For instance, the `inst2ring` and `site_def` options only apply to the VCLEF-fp command, and the `libname` option only applies to Synopsys and TLF commands.

When applying multiple views on your command line, you must specify the view-specific options in detail, as in the following example:

```
% ra2sh <view_command_1> <view_command_2>
  -<view_command_1>.<view-specific_option_1>
  -<view_command_2>.<view-specific_option_2>
```

For example, to generate both Synopsys and VCLEF-fp models, and apply the `inst2ring` and `libname` options, type:

```
% ra2sh synopsys vclef-fp -vclef-fp.inst2ring pins
  -synopsys.libname userlib
```

View commands, descriptions, and output files are listed in Table 2-2.

Table 2-2. View Commands

<View_Command>	Description	Output File(s)
postscript	A-size PostScript datasheet	<instance_name>.ps
ascii	ASCII datatable	<instance_name>.dat
verilog	Verilog model	<instance_name>.v
vhdl	VHDL model	<instance_name>.vhdl
synopsys ^{1, 2}	Synopsys model for each corner	<instance_name>_slow_syn.lib <instance_name>_typical_syn.lib <instance_name>_fast_syn.lib
primitime ²	PrimeTime model for each corner	<instance_name>.mod <instance_name>_slow.data <instance_name>_typical.data <instance_name>_fast.data
tlf ^{1, 2}	TLF model for each corner	<instance_name>_slow.tlf <instance_name>_typical.tlf <instance_name>_fast.tlf
vclef-fp	VCLEF footprint	<instance_name>.vclef <instance_name>_ant.lef <instance_name>_ant.clf
gds2	GDSII layout file	<instance_name>.gds2
lvs	LVS netlist	<instance_name>.cdl <instance_name>.hcell ³
all	All available views	refer to individual View commands
<list>	A list of views, denoted by <list>. ex. ra2sh synopsys ascii	refer to individual View commands

Notes:

- (1) The typical and slow Synopsys and TLF models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.
- (2) Memory generators that have a fourth characterization corner produce additional output files for Synopsys, Primitime, and TLF models. The filename for this fourth output file reflects the parameters of the particular corner. For example, if a generator has an extra fast corner characterized at -40°C, the output files for the Synopsys model will be: <instance_name>_slow_syn.lib, <instance_name>_typical_syn.lib, <instance_name>_fast@-40C_syn.lib, and <instance_name>_fast@0C_syn.lib.
- (3) For more information, see the *Using Hierarchical LVS* section in Chapter 4

Generator View Options

This section describes the syntax used for generator View options you may specify. Details about each option and its default parameter(s) (if any) are provided.

- The Global Options set up the generator for further processing.
- The Generic Options are available with all generators.
- The Advanced Options provide additional choices, depending on the type of generator.
- The View-Specific option specifies library files where needed.

Each of these option types are explained in more detail on the following pages.

Syntax

The syntax described below applies to all options. Use the following syntax for all commands, including as many options as you want to set:

```
ra2sh [-keeplogs] [<View_Command>] [-spec <filename>] [<option>] ...
```

Global Options

-spec *filename*

The specification file contains a list of generic, advanced, and view-specific options and values for the generator. You may create a new specification file, or files, to customize the options that you want to use repeatedly. If you are creating new specification files, be sure to read the "Specification Files" section on page 2-6.

-keeplogs

The default condition creates a new log file each time the generator is run. This new log file overwrites any existing log file. Use this option to keep the existing log file and create a new log file for the current session. For more information, see the "Log File" section on page 2-14.

-help

Links you to this User Guide and any other documents that may have been included for this generator. If the User Guide is not available, a message is displayed. If no other documents are included, the only choice is User Guide.

Generic Options

-instname *name*

The default name is RA2SH. The instance name can be set to any alphanumeric value. To avoid name conflicts for instances within the same library, you must enter a unique instance name. To avoid tool compatibility issues, an instance name of 16 characters or fewer is recommended, and it should begin with a letter. See the additional information in the *-check_instname* and *-prefix* option descriptions.

-words *number*

The default value is 4096. The range for words depends on the multiplexer width and is 128 to 2048 for mux 4, 256 to 4096 for mux 8, 512 to 8192 for mux 16.

-bits *number*

The default value is 16. The range for bits depends on the multiplexer width and is 2 to 128 for mux 4, 2 to 64 for mux 8, 2 to 32 for mux 16.

-frequency *number*

The default value is 1MHz. The frequency can be set to any positive integer value, up to the inverse of the cycle time in nanoseconds (ns) multiplied by 1000. The frequency parameter is used to scale the current-consumption datasheet values. If it is left as the default value of 1.0 megahertz, the units on the current-consumption datasheet values will be milliamperes per megahertz.

`-ring_width number`

The default value is 2 μ m, and is intended as a place holder only. In order for the SRAM to function correctly, you must analyze the ring width requirements based on the design methodology and supply an appropriate ring width value. This value must be a whole number, or integer. For more details, refer to the *Supply Connections to Power Rings* section in Chapter 3.

`-mux 4|8|16`

The default value is 16. The choice for mux is 4, 8, or 16.

`-drive 6`

The default value is 6. The choice for drive strength is 6.

`-write_mask on|off`

The default value is off. The choice for word-write mask is on or off. When the value on is selected, the companion mask options are displayed in the GUI.

`-wp_size number`

The default value is 8. The range for word partition size is 1 to min (36, bits-1) increment = 1.

`-top_layer met4|met5|met6|met7|met8`

The default value is m8. The choice for top metal layer is m4, m5, m6, m7, or m8. For more details, refer to the *Top Metal Layer* section in Chapter 3.

`-power_type rings`

The default value is rings. The choice for power type is rings.

`-horiz met1|met2|met3|met4`

The default value is met3. The choice for horizontal ring layer is met1, met2, met3, or met4.

The horizontal and vertical ring layers may be set to any two sequential metal layers or the same layer (e.g., m1 and m2, or m1 and m1). When a horizontal value is selected, the vertical ring layer is automatically set to a valid value.

`-vert met1|met2|met3|met4`

The default value is met4. The choice for vertical ring layer is met1, met2, met3, or met4.

The horizontal and vertical ring layers may be set to any two sequential metal layers or the same layer (e.g., m1 and m2, or m1 and m1). When a vertical value is selected, the horizontal ring layer is automatically set to a valid value.

Advanced Options

`-customer_comment string`

A customer-specified text field of up to 64 characters. The allowed character set is alphanumeric, plus the following characters: "_", "-", ".", ":", "=", and "+".

The customer comment string will be included in all views. In ASCII text views, it will be inserted as a comment near the top of the file. For more information about the customer comment string, see "Generating Configuration Parameter Files" on page 2-13.

`-left_bus_delim [ | <`

The default value is "[". This option specifies the left bus delimiter for backend views, including VCLEF, GDSII, and LVS. Delimiter choices are "[" or "<". Bus delimiters for front-end views are tool specific, and are not affected by using this option.

`-right_bus_delim ] | >`

The default value is "]". This option specifies the right bus delimiter for backend views, including VCLEF, GDSII, and LVS. Delimiter choices are "]" or ">". Bus delimiters for front-end views are tool specific, and are not affected by using this option.

`-pwr_gnd_rename string`

This option allows you to name the power and ground net names for backend views. An example string, VDD:VCC,VSS:GND, will rename power "VCC" and ground "GND."

`-name_case upper|lower`

The default is upper. This option specifies the case for net names, excluding the top-level instance name and power and ground names.

`-prefix name`

This option allows you to assign a prefix for the instance name. If this option is left blank, no prefix is applied to the instance name. The prefix is counted when using `-check_instname`.

`-diodes on|off`

The default value is on. Because the default value indicates how the generator was validated, the LVS rule set may not support the non-default value. The choice for diodes is on or off.

`-inside_ring_type VDD|GND`

The default value is GND. Defines the inside ring power type as either VDD or GND. The outside ring power type will be of the opposite polarity.

`-pin_space number`

This option specifies the space between the pins of the memory core and the inner power ring segment.

`-check_instname on|off`

The default value is on. The choices for check instance name are on or off. An instance name should begin with a letter, and it should not exceed 16 characters. If this option is turned on and the name does not meet these requirements, the GUI issues error messages, and does not generate views. If the option is turned off, the GUI issues warning messages, but it will generate views. Both errors and warnings display in the log pane, and are recorded in the .log file.

If the generator was launched from the command line and the instance name is greater than 16 characters, the error and warning messages appear on the terminal.

View-Specific Options

For more details about using view-specific options, see "Using View-Specific Options to Generate Multiple View Commands" on page 2-17.

-libname *name*

If you are using Synopsys or TLF, the default library name is USERLIB. Use this option to specify your choice for *libname*. This option applies only to the Synopsys and TLF models.

-inst2ring *pins|blockages*

The default value is blockages. Choose whether the power connections from the memory to the ring wires are modeled as pins or blockages in VCLEF. This option applies only to the VCLEF-fp model.

-site_def *on|off*

The default value is off. This option specifies whether VCLEF contains a site definition. This option applies to only the VCLEF-fp model.



3

***Dual-Port Synchronous SRAM Generator
Architecture***

Synchronous Generator Architecture

The TSMC 0.13-micron synchronous dual-port SRAM generator offers the following features:

- Precise Optimization for TSMC's Eight-Layer Metal 0.13 μ m CMOS Process
- Extremely High Speed
- Fast Access and Cycle Times
- Two Read/Write Ports
- Completely Static Operation
- Near-Zero Hold Time (Data, Address, and Control Inputs)

SRAM Description

The TSMC 0.13-micron synchronous dual-port SRAM is produced by a parameterized block generator which allows great flexibility in the SRAM organization. Three mux options are available: 4, 8, or 16. The SRAM is optimized for TSMC's 0.13-micron, 1.2-volt, eight-layer metal CMOS process technology.

The SRAM has two ports for the same memory locations. Both ports can be independently accessed for read or write operations. In the case of address contention (both ports simultaneously access the same address), both ports will read the same data. Details about the architecture of port A in the RAM is provided below. This explanation is also applicable to port B.

SRAM access is synchronous and is triggered by the rising edge of the clock, CLKA. Input address, input data, write enable, and chip enable are latched by the rising edge of the clock, respecting individual setup and hold times.

The value of chip enable must be low (CENA=0) for a read or write operation to occur. The SRAM enters read mode when the value of chip enable is low and the value of write enable is high. During read mode, data on the data output bus QA[i:0] is read from the memory location specified on the address bus AA[j:0].

If the word-write mask feature is *not* implemented (word-write mask = off), the SRAM enters write mode when the value of chip enable is low (CENA=0) and the value of write enable is low (WENA=0). During write mode, data on the data input bus DA[i:0] is written into the memory location specified on the address bus AA[j:0].

If the word-write mask feature is implemented (word-write mask = on), data on the data input bus DA[i:0] is partitioned to the write enable bus WENA[k:0]. Each WENA[k] pin has a distinct latched value, making each partition individually selectable. When the value of a write enable pin WENA[k] is low, the corresponding data partition is selected, and its data is written to the memory location specified on the address bus A[j:0], and driven through to the data output bus QA[i:0].

For example, a SRAM with 32 bits and a word partition size of 8 (wp_size = 8) will have four write enable pins: WENA[0], WENA[1], WENA[2], and WENA[3]. The write enable pin WENA[0] corresponds to the data partition DA[7:0]; the write enable pin WENA[1] corresponds to the data partition DA[15:8]; the write enable pin WENA[2] corresponds to the data partition DA[23:16]; the write enable pin WENA[3] corresponds to the data partition DA[31:24]. If the values of WENA[0] and WENA[3] are low, and the values of WENA[1] and WENA[2] are high, the data bits DA[7:0] and DA[31:24] will be written to the memory and driven through to the data output bus QA[7:0], QA[31:24]. Even if data is presented on DA[15:8] and DA[23:16], this data is *not* written to the memory and is *not* driven through to the data output bus. Instead, QA[15:8] and QA[23:16] will be the result of a read operation.

Power dissipation is minimized using static circuit implementations. A standby mode is provided to further reduce power dissipation during periods of non-operation (CENA=1). While in standby mode, address and data inputs are disabled; data stored in the memory is retained, but the memory cannot be accessed for reads or writes.

Static power consumption of the SRAM is limited to leakage provided all of the input signals except CLK are held steady.

Block Parameters

The input parameters of a synchronous dual-port SRAM are described in Table 3-1 "Input Parameters".

Table 3-1. Input Parameters

Parameter	Options	
instance name ¹	any alphanumeric name	
number of words	mux = 4	128 to 2048, increment = mux • 2
	mux = 8	256 to 4096, increment = mux • 2
	mux = 16	512 to 8192, increment = mux • 2
number of bits	mux = 4	2 to 128, increment = 1
	mux = 8	2 to 64, increment = 1
	mux = 16	2 to 32, increment = 1
frequency	1 to $1/t_{cyc} \cdot 1000$, increment = 1	
multiplexer width	4, 8, or 16 columns	
drive strength	6	
word-write mask	on or off	
word partition size ²	1 to min (36, bits-1) increment = 1	
top metal layer	met4, met5, met6, met7, or met8	
power type	rings	
ring width ³	2 to 10,000	
horizontal ring layer	met1, met2, met3, or met4	
vertical ring layer	met1, met2, met3, or met4	

Notes:

1. To avoid name conflicts for instances within the same library, you should enter a unique instance name. The instance name can be alphanumeric, but it has to be 16 characters or less.
2. The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. We recommend that the critical path, setup, and hold analysis be performed for all corners.
3. In order for the SRAM to function correctly, you must analyze the ring width requirements based on the design methodology and supply an appropriate ring width. For more details, refer to "Supply Connections to Power Rings" on page 3-12.

Table 3-2 "Secondary Parameters" shows the specifications for a synchronous dual-port SRAM instance.

Table 3-2. Secondary Parameters

Parameter	Options	
total memory bits	256 to 262144 total bits = words • bits	
rows in memory matrix	32 to 512, increment = 2 rows = words / mux	
columns in memory matrix	8 to 512, increment = mux columns = bits • mux	
address lines	mux = 4	7 to 11
	mux = 8	8 to 12
	mux = 16	9 to 13

An example of the physical core mapping for port A for each mux value is shown in Figures 3-2 through 3-4. The physical core mapping for port B is identical to port A. For example, addresses AA_x and AB_x will access the same physical core cell.

Figure 3-2. Mux 4: Core Address Mapping

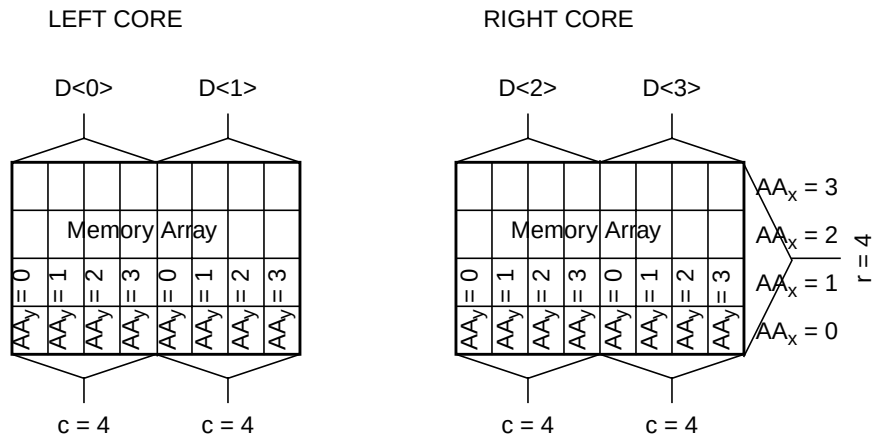


Figure 3-3. Mux 8: Core Address Mapping

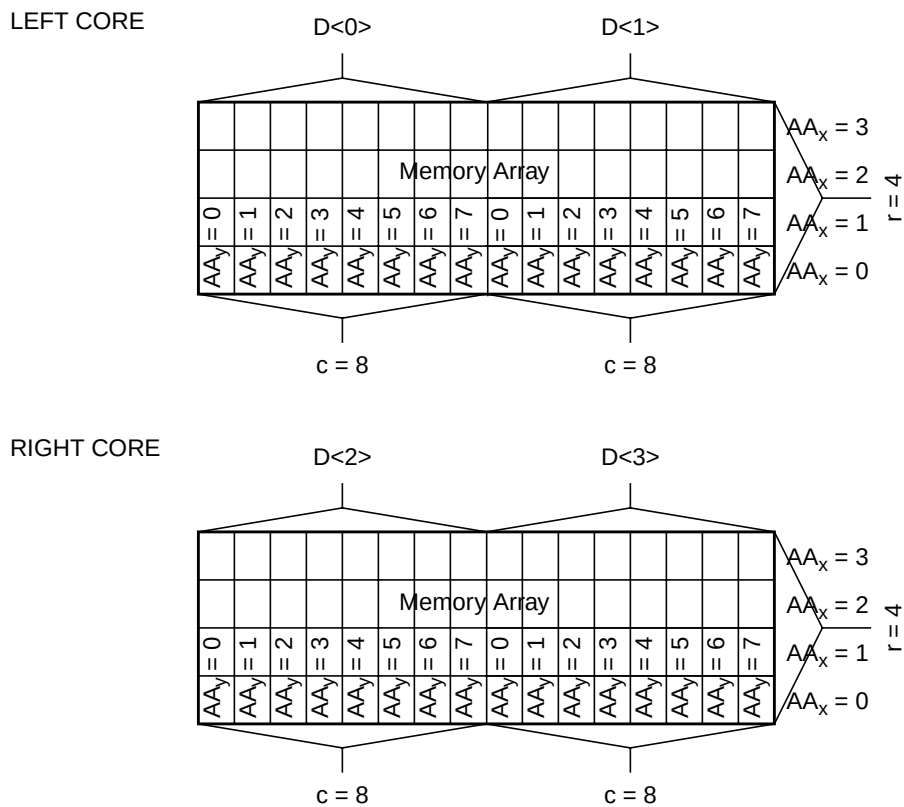
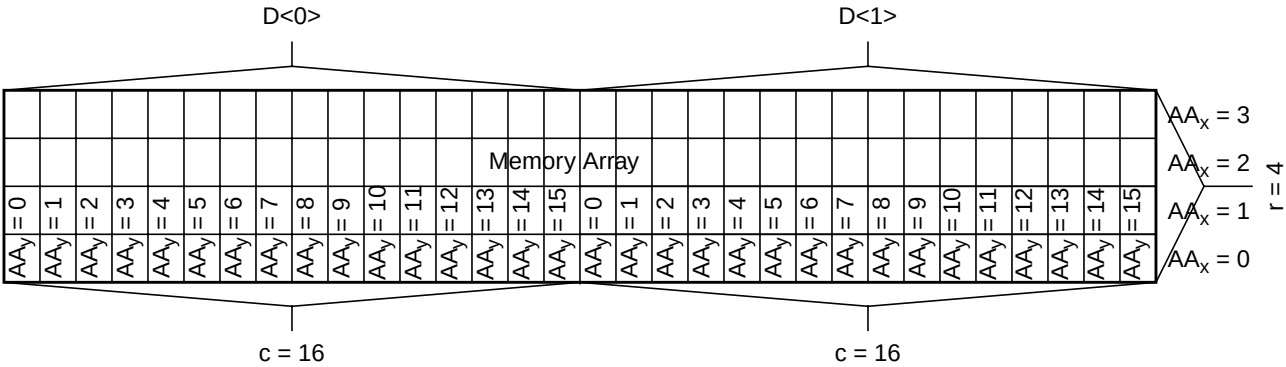
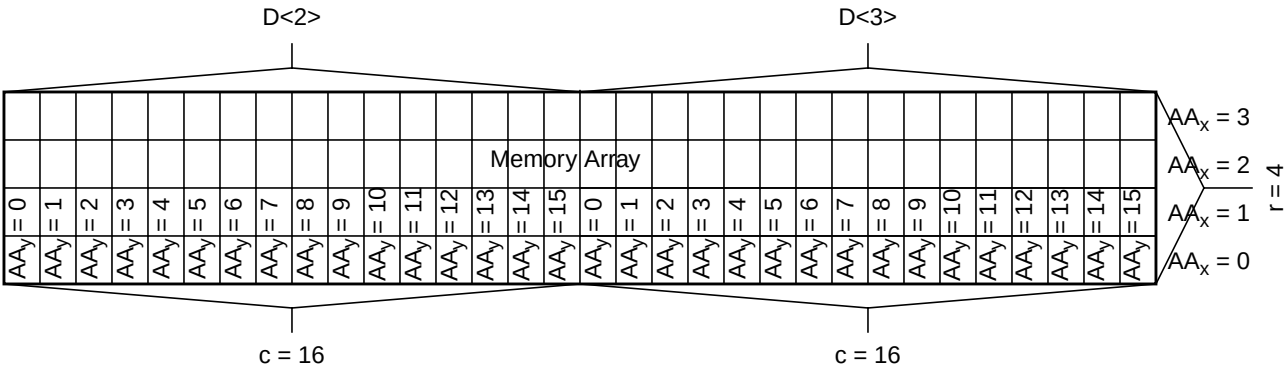


Figure 3-4. Mux 16: Core Address Mapping

LEFT CORE



RIGHT CORE



Power Structure

The following sections detail the power structure options available with the SRAM generator.

SRAM Current

The average current, I_{avg} , in mA, for the SRAM instance is calculated from data reported in the GUI datatable, as well as the datasheet. The average current reported in the datasheet assumes 50% read and write operations where all addresses and 50% of input and output pins [unloaded] switch. You may choose to recalculate this number based on the percentage of reads and writes in a given design. This value is used to calculate the size of the power bus.

Given:

c = average capacitance of output port (pF);

n = number of ports;

From the datatable,

$$I_{avg} = \left(icc + \left(\frac{1}{2} \cdot cvf \cdot bits \right) \right) \cdot n.$$

From the datasheet,

$$I_{avg} = AC \text{ Current} + \left(\frac{1}{2} \cdot cvf \cdot bits \right) \cdot n.$$

The peak current, I_p , in mA, for the instance is reported in the GUI datatable, as well as the datasheet. This peak current is calculated during read/write HSPICE simulations and yields a simulated peak. The amplitude of the peak may be large, but the duration is very short due to ideal circuit behavior.

If the memory is deselected and only the clock switches, then the current is the same as standby current. Standby current assumes no switching, and normal reverse-bias leakage.

 NOTE: When the SRAM is deselected, all addresses switch, and 50% of input pins switch, then current consumption may be up to 30-40% of icc because the input latches are open, and the internal logic can switch. The

logic-switching component of deselected power becomes small if the address, data, and write enable pins are held stable by externally controlling these signals with chip select.

From the datatable,

$$I_p = \text{icc_peak}$$

From the datasheet,

$$I_p = \text{Peak Current}$$

Power Distribution Methodology

The chip-level power distribution must ensure that the wire widths supplied to the SRAM satisfy electromigration guidelines and limit the average and peak voltage drop in the power wires to an acceptable value. To ensure memory timing accuracy, the effective voltage supplied to the memory must be the same as the characterized voltage. You must properly size the supply wire widths. The SRAM minimum supply wire widths are calculated as follows.

For example, given:

W_{em} = connection width based on electromigration (μm);

W_{iravg} = connection width based on average voltage (μm);

W_{irp} = connection width based on peak voltage (μm);

C = current density rule constant ($\text{mA}/\mu\text{m}$);

I_{avg} = average current consumed by the SRAM (mA);

I_p = peak current consumed by the SRAM (mA);

ΔV_{iravg} = allowable average voltage drop within the power wires on the chip (mV);

ΔV_{irp} = allowable peak voltage drop within the power wires on the chip (mV);

L_{eff} = effective wire length of power connection from power pad to the SRAM (μm);

R_m = resistance of metal wire (Ohms/square);

W = connection width (μm);

we have:

$$W_{em} = \frac{I_{avg}}{C},$$

$$W_{iravg} = \frac{L_{eff} \cdot R_m}{\Delta V_{iravg}} \cdot I_{avg}$$

$$W_{irp} = \frac{L_{eff} \cdot R_m}{\Delta V_{irp}} \cdot I_p$$

$$W = \max(W_{em}, W_{iravg}, W_{irp})$$

 These sample calculations do not take into account the other components on the chip that may be supplied by the same wire. You must adjust wire width accordingly. The L_{eff} parameter can also be adjusted to account for the varying width of the power wires.

Supply Connections to Power Rings

The generator has the capability of generating power rings around the SRAM. You must properly size these rings. The size of the rings depends on the chip-level power distribution methodology, the number, width, and placement of supply wire connections to the power rings, and current consumption.

Artisan recommends that the current be evenly supplied from the edge of the instance where the I/O pins are located.

In the case of one supply wire connection to the SRAM power rings, the ring width must be at least half the width of the power bus connection. This is possible if the current is approximately equally distributed on either side of the connection into the ring.

In case of multiple connections to the ring, the width may be reduced only if the connections are evenly distributed along the edge where the I/O pins are located. The ring width must remain equal to or greater than half the width of the widest power connection.

Given:

W_r = ring width with one power connection (μm);

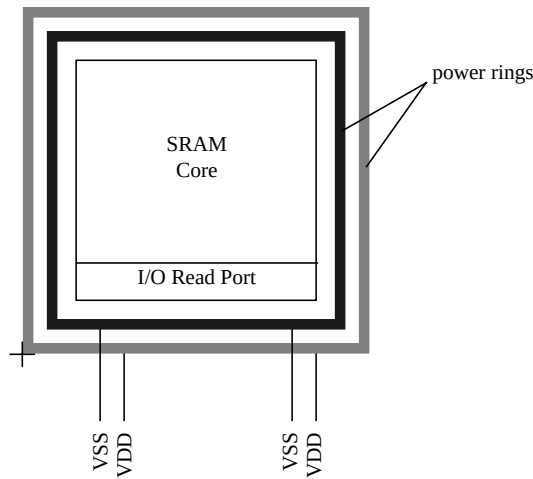
n = number of equal-width-wire power connections at the I/O pins;

we have:

W_m = ring width with n connections (μm) = $1/n \bullet W_r$

Figure 3-6 shows I/O connections on the bottom edge of the SRAM block.

Figure 3-5. Multiple Power and Ground Connections



Top Metal Layer

The generator includes the capability of specifying the top-most metal layer utilized by the SRAM. For example, a process may support a maximum of eight layers but you may elect to use only five layers for a given chip design. The layout size is the same for all top metal layer options.

All metal layers below and including metal4 are used in the design, and therefore, they are blocked. All metal layers above metal4 can be routed over the memory.

I/O Connections

Input/output pins are located along the top and bottom edge of the memory block on any of the metal layers. The I/O pins are large enough to accommodate an on-grid 0.2-micron width wire connection, and the pins are designed to be on the grid even when the memory is rotated or placed off the grid. (This assumes that the routing grid is less than or equal to 0.46 micron.) See Figure 3-6 for reference. Depending on the chip-level placement of a memory instance, a pin geometry may enclose multiple grid points but, as a worst case, only one is valid. (A valid grid point is enclosed by the pin geometry by at least half of a wire width, 0.1 micron.) The router must access the pin by way of the routing track that corresponds to a valid grid point and is perpendicular to the cell edge. If the router approaches the pin off-grid and then bends the wire underneath the obstruction layer to connect to the valid grid point, a metal spacing or short circuit error may result.

For added flexibility, the pins can be accessed on several different routing layers. For example, pins can be accessed on metal1, metal2, metal3, and metal4. In most cases, the pin access layer is determined by the horizontal power ring layer, the memory orientation, and the chip-level routing methodology. The I/O wires must route across the horizontal power ring layer and therefore cannot be the same layer.

Timing Specifications

This section contains the pin descriptions, timing diagrams, and the specifications for the timing parameters of the TSMC 0.13-micron synchronous dual-port SRAM.

Table 3-3 "Pin Description" describes the pins.

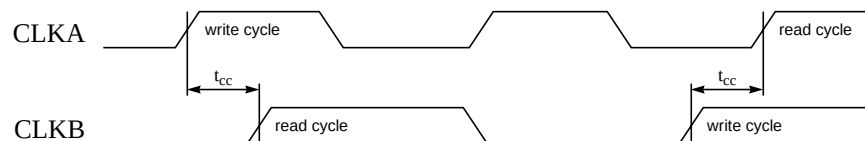
Table 3-3. Pin Description

Pin	Description
AA[i]	Addresses (AA[0] = LSB)
AB[i]	Addresses (AB[0] = LSB)
DA[i]	Data Inputs (DA[0] = LSB)
DB[i]	Data Inputs (DB[0] = LSB)
CLKA, CLKB	Clock Inputs
CENA, CENB	Chip Enables
WENA[k], WENB[k]	Write Enables
QA[i]	Data Outputs (QA[0] = LSB)
QB[i]	Data Outputs (QB[0] = LSB)

Clock Timing

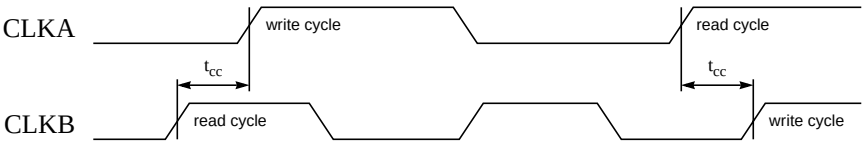
The TSMC 0.13-micron synchronous SRAM clock timing diagrams are shown below.

Figure 3-6. Write-Read Clock Timing (Accessing Same Address)



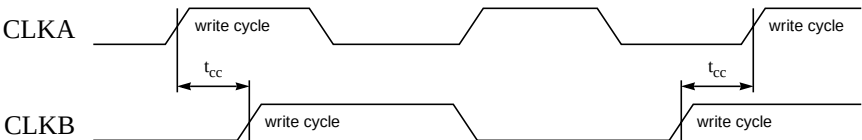
Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-7. Read-Write Clock Timing (Accessing Same Address)



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-8. Write-Write Clock Timing (Accessing Same Address)



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Table 3-4 illustrates read and write behavior during clock contention, when both ports access the same address.

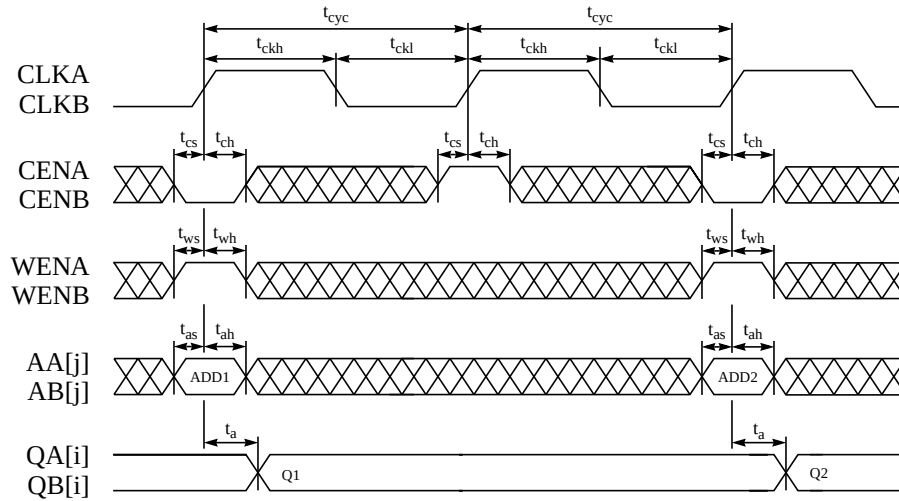
Table 3-4. Read and Write Behavior When Accessing Same Address

Action	Condition	Behavior
write from one port then read from the other port	t_{cc} is satisfied (see Figure 3-7)	write OK D-to-Q write through OK read (new data) OK
	t_{cc} is not satisfied (see Figure 3-7)	write OK D-to-Q write through OK read fails
read from one port then write from the other port	t_{cc} is satisfied (see Figure 3-8)	write OK D-to-Q write through OK read (old data) OK
	t_{cc} is not satisfied (see Figure 3-8)	write OK D-to-Q write through OK read fails
write from one port then write from the other port	t_{cc} is satisfied (see Figure 3-9)	both writes OK (second write overwrites first write) D-to-Q write throughs OK
	t_{cc} is not satisfied (see Figure 3-9)	both writes fail D-to-Q write throughs OK
read from one port then read from the other port	no restriction	both reads OK

Mission-Mode Timing

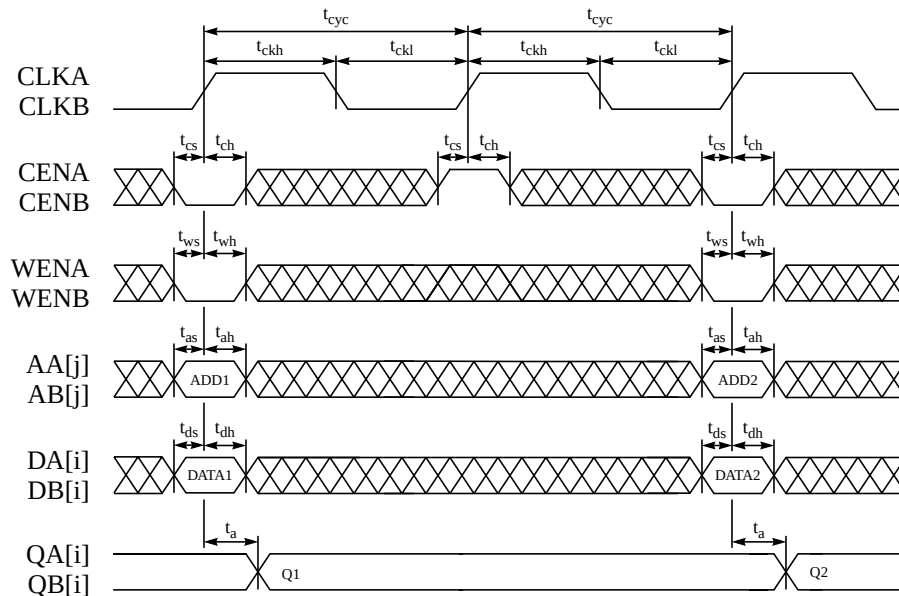
The TSMC 0.13-micron synchronous SRAM mission-mode timing diagrams are shown below.

Figure 3-9. Read-Cycle Timing

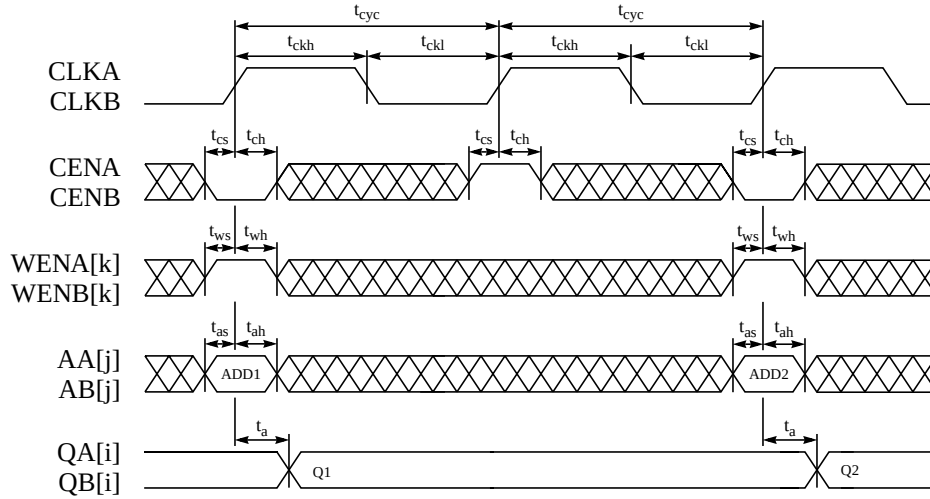


Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

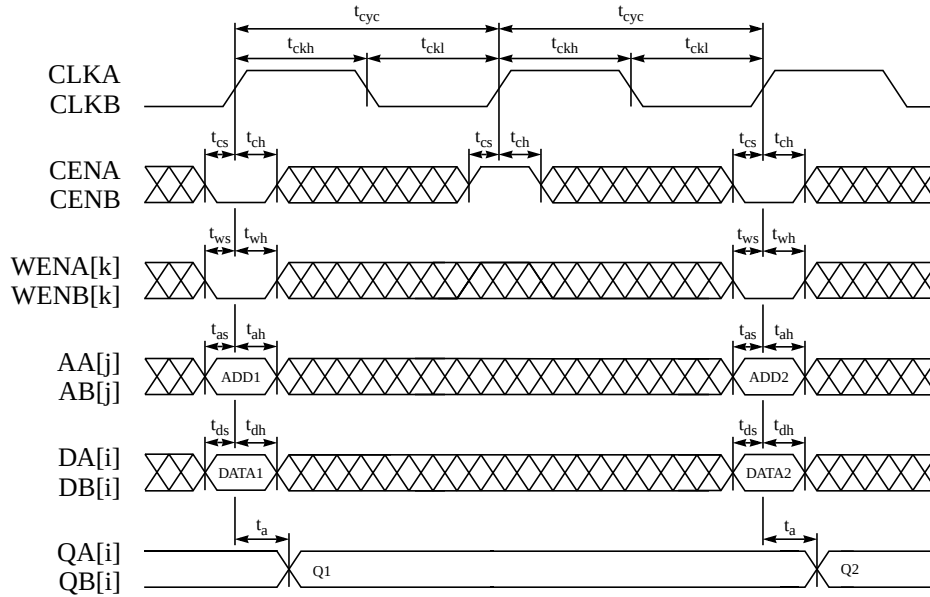
Figure 3-10. Write-Cycle Timing



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-11. Read-Cycle Timing, with Word-Write Mask

Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-12. Write-Cycle Timing, with Word-Write Mask

Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Timing Parameters

The datasheets will contain the timing parameters given in Table 3-5.

Table 3-5. Mission-Mode Timing Parameters

Parameter	Symbol
Cycle time	t_{cyc}
Access time ^{1,2}	t_a
Address setup	t_{as}
Address hold	t_{ah}
Chip enable setup	t_{cs}
Chip enable hold	t_{ch}
Write enable setup	t_{ws}
Write enable hold	t_{wh}
Data setup	t_{ds}
Data hold	t_{dh}
Clock high	t_{ckh}
Clock low	t_{ckl}
Clock collision	t_{cc}

Notes:

¹ Parameters have a load dependence (K_{load}), which is used to calculate:

$$\text{TotalDelay} = \text{FixedDelay} + (K_{load} \times C_{load}) .$$

² Access time is defined as the slowest possible output transition for the typical and slow corners, and the fastest possible output transition for the fast corner.

Characterization Environments

The characterization environments for the TSMC 0.13-micron synchronous dual-port SRAM are specified in Figure 3-6.

Table 3-6. Characterization Environments

Corner	Process	Temperature	Voltage
fast	fast pmos fast nmos	-40°C	1.32V
-0	fast pmos fast nmos	0°C	1.32V
typical	typical pmos typical nmos	25°C	1.2V
slow	slow pmos slow nmos	125°C	1.08V

Timing Derating

Process-Perfect™ memories do not support a timing derating methodology. Timing is provided for four characterization environments: fast (1.32V, -40°C), -0 (1.32V/0°C), typical, and slow. Refer to Table 3-6 "Characterization Environments" for details.

Several delay calculators and the associated timing views, such as Synopsys and TLF, include a simplistic derating ability and a specified derating factor. The derating methodology supported by these delay calculators and the specified derating factor is not sufficient to accurately model the timing behavior of the memory. The specified derating factor is a general derating factor that was derived using simple gate delays. Actual memory timing results, when compared to results using this derating factor, can be significantly optimistic or pessimistic, depending on the size of the instance and the voltage and temperature change from the reference timing model.

Relying on timing results using derating may lead to memory timing constraint violations and may cause a non-working part.



4

Using the Dual-Port SRAM Generator Views

Using the Generated Views

Tool Verification

The TSMC 0.13-micron dual-port SRAM generator produces views, or models, that have been verified with the tools shown in Table 4-1.

Table 4-1. Tool Verification

Model	Verified Tool
Verilog	Cadence's NC-Verilog 3.1
VHDL	Model Technology's ModelSim 5.5
Synopsys	Design Compiler 2000.11
TLF 4.1	Cadence's Pearl 4.3-s086
PrimeTime	Synopsys' PrimeTime 2001.06

Using the Verilog Model

After generating the Verilog model, simulate the design module using these steps.

Check the syntax of the Verilog model:

```
verilog <name>.v
```

where <name>.v is the Verilog model.

Simulations can be done in your test bench where the model is instantiated. For functional simulations:

```
verilog +define+functional <test-bench>
```

Use the SDF annotator to back-annotate the SDF timing files to the verilog model. An example command is:

```
$sdf_annotate(<sdf_file_name>, <instance>)
```

Run the simulation:

```
verilog <test-bench>.v
```

The simulation output is written to a file, *verilog.log*, which is placed in the current directory.

Using the VHDL Model

After generating the VHDL model, simulate the design module using these steps.

The *work* library is necessary for the compilation of any VHDL model. If it does not already exist, create the *work* library:

```
vlib work
```

Compile the Artisan library file (only once):

```
vcom <install_dir>/aci/ra2sh/lib/vhdl_lib/  
artisan_lib.vhd
```

Compile the VHDL design:

```
vcom <name>.vhd
```

where *<name>.vhd* is the VHDL model.

Define a test bench in which the design unit is instantiated. It can be referenced through *<name>_pkgs* by using the clause:

```
use work.<name>_pkgs.all;
```

in the test bench file, where *<name>_pkgs* is a package included in the generated VHDL model.

Compile your test bench:

```
vcom <test-bench>.vhd
```

After compilation of the test bench, view the simulation results:

```
vsim
```

In the *Startup* pop-up window, highlight the module and the architecture bound to the design entity.

Click on the *Load* button. After loading the design entity, a *VSIM* pop-up window appears. From the *Run* pull-down menu, select *Time High* to run simulations for the maximum time specified in the test bench.

The simulation results can be viewed as waveforms or ascii text. From the *View* pull-down menu in the *VSIM* window, select *Signals*.

To view the waveforms of the signals in the design, select the *Wave* pull-down menu in the *VSIM Signals* pop-up window, and then select *Signals in Design*. Next, select the *View* pull-down menu in the *VSIM* window, and select *Waves*. The *VSIM Wave* window shows the waveforms. To save the configuration, select the *File* pull-down menu and then select *Save Configuration*. By default, the configuration is saved into the *wave.doc* file which is placed in the current directory. To print the waveforms, select the *File* pull-down menu and then select *Write Postscript*.

To view the simulation of the design as ascii text, select the *List* pull-down menu in the *VSIM Signals* pop-up window, and then select *Signals in Design*. Next, select the *View* pull-down menu in the *VSIM* window, and select *List*. The *VSIM List* window shows the ascii text. To save to a file, select the *File* pull-down menu and then select *Write to File*. By default, the list is saved into the *vsim.ls* file which is placed in the current directory.

The VHDL model can be back annotated into SDF:

```
vcom -sdf<corner> <name>.sdf
```

where *<name>.sdf* is the output SDF file and *<corner>* is *min*, *typ*, or *max*.

Using the Synopsys Model to Generate SDF

After generating the Synopsys model, you can generate SDF using these steps.

Invoke Synopsys:

```
dc_shell
```

Once inside `dc_shell`, execute the following Synopsys commands:

```
read_lib <name>.lib
write_lib <userlib>
link_library=<userlib>.db
target_library=<userlib>.db
read -f verilog <file>.v
write_timing -context verilog -f sdf-v2.1 -o <out>
```

where `<userlib>` is the name of your Synopsys library, `<name>.lib` is the Synopsys model, `<file>.v` is the top-level netlist, and `<out>` is the output file name.

If you are using multiple Synopsys libraries, you can read the `.lib` files into the Synopsys model and include each file in your `link_library` and `target_library` paths. You can write a script or manually remove the `lu_table_template` descriptions, `power_lut_template` descriptions, `type` descriptions, and `cell()` block for each `.lib` file. You should include all the descriptions and block into a single `.lib` file, and append the global library information found at the beginning of the generated `.lib` files to the beginning of your consolidated `.lib` file. Attach a closing bracket `"}"` to the end of the new `.lib` file.

In the Synopsys model, data from SPICE characterization is used for setup and hold times; no negative setup is used in the SRAM. In SDF, if the hold time is a negative number, it is set to zero.



Artisan recommends that you avoid using implicit netlisting because it is an error prone methodology.

Using the Pearl Model to Generate SDF

After generating the Pearl model, you can generate SDF using these steps.

Invoke the Pearl Timing Analysis tool:

```
pearlcell
```

Once in Pearl, execute the following commands:

```
ReadTechnology <your_technology_file>
Read TLF <name>.tlf
Read Verilog <your_verilog_file>
TopLevelCell <your_toplevel>
InputSlew <toplevel_input_pin_name>
           <value> <value> <value> <value> <value>
.
.
.
SetNodeCapacitance <toplevel_output_pin_name> + <value>
.
.
.
WriteSDFDelays -delimiter . -version2.1 -ns -precision 3
               "<your_sdf_file>"
quit
```

Using the PrimeTime Model to Generate SDF

After generating a Synopsys .db file, you can generate SDF using these steps.

Invoke PrimeTime:

```
pt_shell
```

Once inside pt_shell, execute the following PrimeTime commands:

```
read_db <name>.db  
set link_path=<userlib>.db  
read_verilog <file>.v  
write_sdf -version 2.1 -o <out>
```

where <userlib> is the name of your Synopsys library, <name>.lib is the Synopsys model, <file>.v is the top-level netlist, and <out> is the output file name.

Loading the VCLEF Description into Silicon Ensemble

After generating VCLEF, load it into Silicon Ensemble using these steps.

Invoke Silicon Ensemble.

In the Command window, type:

```
INPUT LEF FILENAME "<path>/<name>.vclef";
```

where *<path>* is the path to the VCLEF, and *<name>* is the SRAM instance name.

This creates the Silicon Ensemble database necessary for routing.

Using Apollo with Artisan SRAM

In order to use the Artisan Components SRAM with the Avant! tool suite you need to import the SRAM into the Milkyway database. Before you can place or route a design you need to create a FRAM view for any memories in the design. This can be done by importing the VCLEF and running "Blockage, Pin & Via" (BPV) to create the FRAM. If you wish to stream out a full GDSII database you also need to import the gdsII into the Milkyway database to create a CEL view.

Artisan does not recommend trying to create a FRAM view directly from the gdsII. You must use the following sequence when importing SRAM into the Milkyway database to avoid creating a FRAM view from the gdsII.

1. Generate the VCLEF and GDSII
2. Import the VCLEF into Milkyway
3. Run BPV to generate a FRAM view
4. Import the GDSII into Milkyway

It is important to run BPV before importing the GDSII.

Details on creating and importing VCLEF and GDSII are provided in the following sections. Consult the Avant! documentation for more details on the commands mentioned below. In particular, you should be familiar with Chapter Four of the Avant! *Milkyway Data Preparation User Guide*.

Loading the VCLEF Description into the Milkyway Database

Create a LEF or VCLEF file from the generator.

Example:

```
<install-dir>/ra2sh/bin/ra2sh vclef-fp  
-words 256 -bits 16 -mux 8 -instname <name>
```

This command creates a file called `<name>.vclef`. The instance name for this file is `<name>`.

Edit `<name>.vclef` and comment the line that contains "VERSION 5.1". Commented lines in LEF begin with the pound sign (#). Create a file named `lef2Arcs.map` using the same syntax as the `gds2Arcs.map` file.

Example:

```
gdsMacroCell  
<name>
```

Start Milkyway. On the command line for Milkyway, use the Avant! undocumented LEF reader `auLefIn`.

A form displays. Fill this form with the appropriate entries for library name, .lef file name, and `lef2Arcs.map` file, then click on OK to create the CEL view.

Now run Blockage, Pin & View (BPV) to create the FRAM view.

Loading the GDSII Layout into the Milkyway Database

Create a GDSII file from the generator.

Example:

```
<install-dir>/ra2sh/bin/ra2sh gds2  
-words 256 -bits 16 -mux 8 -instname <name>
```

This command creates a file called `<name>.gds2`. The instance name for this file is `<name>`.

Create a file named `gds2Arcs.map`.

Example:

```
gdsMacroCell  
<name>
```

Start Milkyway. The VCLEF should have already been read into the library created in the previous section. Read in the GDSII. This process overwrites the CEL view with the actual layout. From the menus, select **Cell Library > Stream in...** Fill in the **Stream File Name** and the **Library Name**. Depending on your flow, the other fields may or may not need to be updated.

Loading the GDSII Layout into a DFII Library

After generating the GDSII layout, you can load it into a TSMC 0.13-micron DFII library using these steps.

Invoke DFII.

From the DFII CIW, select the *File* pull-down menu, then select the *Import* pull-down sub menu and click on *Stream*.

In the *Stream* pop-up window, type the name of the GDSII layout file in the *Input File* field, and type the SRAM instance name in the *Top Cell Name* field. Type the name of the library in the *Library Name* field. Click on *User-Defined Data*.

In the *User-Defined Data* pop-up window, type the path to the metal layer table file in the *Layer Map Table* field, and type the path to the text font file in the *Text Font Table* field.

Using the LVS Netlist

After generating the LVS netlist, it may be used in conjunction with the GDSII file for verification.

Typically, you use a tool like Cadence Dracula, Mentor Graphics Calibre, or Avant! Hercules, to read a GDSII file and compare it with the LVS netlist. This test compares the layout and schematic to ensure there is no short- or open-circuit in the layout.

The LVS netlist is then added onto the chip level LVS netlist, and the same test is run when the chip is fully assembled. This process ensures that the chip is correctly assembled, i.e. there is no short- or open-circuit caused by a place-and-route or other tool.

Using Hierarchical LVS

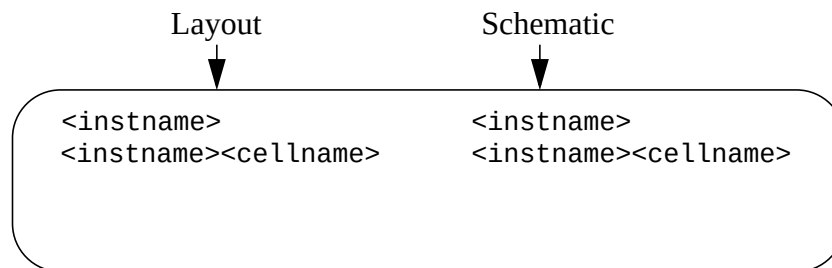
LVS (Layout vs. Schematic) performs an equivalence check between two different representations of a design. In this case, the physical (GDSII) and schematic (SPICE netlist) are compared to each other. The CALIBRE tool reports any discrepancies.

Executing LVS in a hierarchical mode is faster than using LVS in the flat mode. The blocks are checked only once, as opposed to multiple passes for various instantiations of the same block in the design.

To run calibre LVS on memory instances in full hierarchical mode, execute the following steps:

1. Invoke the SRAM compiler GUI.
2. Generate the GDSII and LVS netlists to create the `<instname>.gds2` and `<instname>.cdl`.
3. Create the `.hcell` file.

Sample Hcell file format:



The left column corresponds to the layout instances and the right column corresponds to schematic instances where:

`<instname>` is the instance name you specified when the gds2 and cdl views were created

`<cellname>` is a hierarchical cell within the SRAM

4. Modify the rules file.

The rules file specifies the location and format of the following items:

```
LAYOUT PATH "<instname>.gds2"  
LAYOUT PRIMARY <instname>  
SOURCE PATH "<instname>.cdl"  
SOURCE PRIMARY <instname>  
LVS REPORT "<instname>.lvs"
```

5. Execute CALIBRE:

```
calibre -lvs -hier -spice <instname>.spc  
-hcell <instname>.hcell rulesfile
```

where the .spc file is output from calibre.

6. Examine the LVS report.

Troubleshooting Notes:

More information about hierarchical LVS can be found in Mentor Graphics' ***Physical Extraction and Verification Application Note #21: What to look for in the Calibre LVS transcript.***

Specifically, review the section on the command "LVS SHOW SEED PROMOTIONS YES"

This information is available online at Mentor Graphics' web site.

Index

A

- advanced options
 - list [2-22](#)
 - menu [2-8](#)
 - using [2-8](#)
- Apollo, using with Artisan products
 - GDSII [4-12](#)
 - VCLEF [4-11](#)
- Artisan
 - generator GUI [2-4](#)
 - website [vii](#)
- ASCII datatable
 - output filename [2-18](#)
 - units [2-16](#)
 - using [2-15](#)
- Avant! tool suite [4-11](#)

B

- bus delimiter
 - left [2-22](#)
 - right [2-22](#)

C

- CALIBRE [4-15](#)
- check instance name option [2-23](#)
- core address mapping [3-7](#)
- corners [3-21](#)
- current
 - average [3-10](#)
 - deselected [3-10](#)
 - peak [3-10](#)
 - standby [3-10](#)
- customer service [vii](#)

D

- datasheet
 - generate [2-12](#)
 - postscript output file [2-18](#)
- datatable, ASCII [2-15](#)
- delays
 - Synopsys [2-9](#)
 - TLF [2-9](#)
- diodes option [2-23](#)
- directory
 - installation [1-4](#)
 - path [2-4](#)
 - structure [1-6](#)
 - working [2-4](#)
- document symbols [viii](#)
- drive strength option [2-21](#)

E

- environment
 - fast [3-21](#)
 - slow [3-21](#)
 - typical [3-21](#)

F

- footprint [2-16](#)
- frequency option [2-20](#)

G

- GDSII layout [4-13](#)
- generating
 - all views [2-10](#)
 - configuration parameter files [2-13](#)
 - postscript datasheet [2-12](#)
- generic options [2-20](#)
- global options [2-19](#)

graphic user interface (GUI) [2-4](#)

GUI

- ASCII datatable [2-15](#)
- generic parameters pane [2-7](#)
- relative footprint [2-16](#)
- screen image [2-5](#)
- using [2-4](#)

H

- help display [2-20](#)
- hierarchical LVS [4-15](#)
- horizontal ring layer option [2-21](#)

I

- installation directory [1-4](#)
- instance name
 - check [2-23](#)
 - option [2-20](#)
 - prefix option [2-23](#)
 - using [3-5](#)
- instance to ring wires option [2-24](#)

L

- library name
 - option [2-24](#)
 - Synopsys [2-9](#)
 - TLF [2-9](#)
- LVS netlist [4-14](#)

M

- menu
 - generate [2-10](#)
 - utilities [2-6](#)
 - views [2-8](#)

- models, using
 - hierarchical LVS [4-15](#)
 - Pearl [4-8](#)
 - PrimeTime [4-9](#)
 - Synopsys [4-7](#)
 - Verilog [4-4](#)
 - VHDL [4-5](#)

- multiplexer width option [2-21](#)
- mux option [2-21](#)

N

- name case option [2-23](#)

O

- operating system, requirements [1-4](#)
- option
 - bus delimiter, left [2-22](#)
 - bus delimiter, right [2-22](#)
 - check instance name [2-23](#)
 - diodes [2-23](#)
 - drive strength [2-21](#)
 - frequency [2-20](#)
 - help [2-20](#)
 - horizontal layer [2-21](#)
 - instance name [2-20](#)
 - instance name prefix [2-23](#)
 - instance to ring wires [2-24](#)
 - library name [2-24](#)
 - multiplexer width [2-21](#)
 - name case [2-23](#)
 - number of bits [2-20](#)
 - number of words [2-20](#)
 - pin space [2-23](#)
 - power type [2-21](#)
 - power/ground rename [2-22](#)
 - ring width [2-21](#)
 - site definition [2-24](#)

- specification file [2-19](#)
- top layer metal [2-21](#)
- vertical layer [2-22](#)
- word partition size [2-21](#)
- word write-mask [2-21](#)

options

- advanced [2-22](#)
- generic [2-20](#)
- global [2-19](#)
- view-specific [2-24](#)

P

- Pearl model [4-8](#)
- pin space option [2-23](#)
- postscript datasheet [2-12](#)
- power and ground rename option [2-22](#)
- power distribution methodology [3-11](#)
- power ring
 - size [3-5](#), [3-12](#)
 - supply connections [3-12](#)
- power type option [2-21](#)
- PrimeTime model [4-9](#)

R

range

- address lines [3-6](#)
- bits [3-5](#)
- columns in memory matrix [3-6](#)
- drive strength [3-5](#)
- frequency [3-5](#)
- horizontal ring layer [3-5](#)
- multiplexer width [3-5](#)
- power type [3-5](#)
- ring size [3-5](#)
- rows in memory matrix [3-6](#)
- top metal layer [3-5](#)
- total memory bits [3-6](#)

- vertical ring layer [3-5](#)
- word partition size [3-5](#)
- words [3-5](#)
- word-write mask [3-5](#)

- relative footprint [2-16](#)
- requirements, operating system [1-4](#)
- revision history [vii](#)
- ring width option [2-21](#)

S

SDF

- Pearl model [4-8](#)
- PrimeTime model [4-9](#)
- Synopsys model [4-7](#)
- Silicon Ensemble, loading VCLEF [4-10](#)
- site definition option [2-24](#)
- specification file
 - creating [2-6](#)
 - option [2-19](#)
 - using [2-6](#)
- SRAM current [3-10](#)
- Synopsys model [4-7](#)

T

- technical support [vii](#)
- temperature [3-21](#)
- text symbols [viii](#)
- timing
 - derating [3-21](#)
 - diagrams [3-15](#)
 - parameters [3-20](#)
- timing derating [3-21](#)
- top metal layer [3-14](#)
 - option [2-21](#)
- typographic conventions [viii](#)

U

utilities menu [2-6](#)

V

VCLEF [4-10](#)

Verilog model [4-4](#)

vertical ring layer option [2-22](#)

VHDL model [4-5](#)

view commands

 output file location [2-17](#)

 output filenames [2-18](#)

view-specific option [2-24](#)

voltage [3-21](#)

W

word partition

 input pin capacitance [3-5](#)

 size [3-5](#)

 size option [2-21](#)

word write-mask option [2-21](#)

working directory [2-4](#)